Universidad Rafael Landívar
Facultad de Ingeniería
Inteligencia Artificial
Primer Semestre 2026

Proyecto: Clasificación de Solicitudes a Mesa de Ayuda

Clasificación de Texto usando Naïve Bayes Multinomial

Christopher Yuman
Susana García
Erick Rivas

Guatemala, 16 de abril de 2026

Índice

1. Introducción *4*

2. Definición del problema y objetivos *5*

2.1 Definición del problema 5

2.2 Objetivo general 5

2.3 Objetivos específicos 5

3. Descripción del dataset utilizado *6*

3.1 Categorías del dataset 6

4. Descripción del preprocesamiento aplicado *8*

4.1 Limpieza del texto 8

4.2 Tokenización 8

4.3 Eliminación de stopwords 8

4.4 Stemming 8

4.5 Construcción del vocabulario (Bag of Words) 8

4.6 Resumen del pipeline 9

5. Explicación matemática del algoritmo Naïve Bayes *10*

5.1 Fundamento del algoritmo 10

5.2 Modelo Multinomial 10

5.3 Probabilidad a priori 10

5.4 Suavizado de Laplace (Laplace Smoothing) 10

5.5 Suma de logaritmos 11

6. Explicación e interpretación de las métricas de evaluación *12*

6.1 Precisión (Precision) 12

6.2 Recall 12

6.3 F1-Score 12

6.4 Accuracy (Exactitud global) 12

6.5 Macro F1 12

6.6 Matriz de Confusión 12

6.7 Resultados por clase 13

6.8 Matriz de Confusión 13

6.9 Análisis crítico 13

7. Resultados obtenidos con K-Folds y análisis de varianza *15*

7.1 Validación cruzada con K=5 15

7.2 Análisis de varianza entre folds 15
8. Diagramas *16*
8.1 Diagrama de Arquitectura de la Solución 16
8.2 Diagrama de Casos de Uso 16
8.3 Diagrama de Flujo General 16
8.4 Diagrama de Componentes 18
8.5 Diagrama de Secuencias 18
9. Evidencias de funcionamiento *20*
9.1 Salida del entrenamiento y evaluación 20
9.2 Interfaz web 20
9.3 Ejemplo de clasificación 22
10. Conclusiones y aprendizajes *23*
10.1 Conclusiones 23
10.2 Aprendizajes 23

1. Introducción

En el ámbito del servicio al cliente, las empresas reciben diariamente grandes volúmenes de solicitudes que deben ser dirigidas al departamento adecuado para su resolución. Este proceso, cuando se realiza de forma manual, resulta lento, propenso a errores y costoso en términos de recursos humanos. La automatización de esta tarea mediante técnicas de clasificación de texto representa una solución eficiente y escalable.

El presente proyecto tiene como propósito construir un sistema de clasificación automática de solicitudes de soporte al cliente utilizando el algoritmo Naïve Bayes Multinomial, implementado completamente desde cero en Python. El sistema es capaz de recibir el texto de una solicitud y asignarle automáticamente una de once categorías predefinidas, tales como ORDER, BILLING, SHIPPING, REFUND, ACCOUNT, entre otras.

A diferencia de aproximaciones que emplean bibliotecas de machine learning como scikit-learn o PyTorch, este proyecto enfatiza la comprensión e implementación manual de cada etapa del algoritmo: desde el preprocesamiento del texto, la construcción del vocabulario mediante Bag of Words, el cálculo de probabilidades con suavizado de Laplace, el uso de logaritmos para evitar problemas de underflow numérico, hasta la evaluación rigurosa del modelo mediante validación cruzada K-Folds y el análisis detallado de las métricas obtenidas.

Además del motor de inferencia, el proyecto incluye una aplicación web funcional desarrollada con FastAPI que permite ingresar solicitudes en tiempo real y recibir la categoría predicha, simulando así un sistema real de enrutamiento automático de tickets de soporte técnico.

2. Definición del problema y objetivos

2.1 Definición del problema

Las empresas que ofrecen productos o servicios al público reciben constantemente solicitudes de soporte a través de diferentes canales. Cada solicitud debe ser categorizada y dirigida al departamento correspondiente para su atención. Este proceso de clasificación manual consume tiempo, introduce errores humanos y genera cuellos de botella cuando el volumen de solicitudes aumenta.

El problema consiste en construir un clasificador de texto que, dado el contenido de una solicitud de cliente, determine automáticamente a cuál de las once categorías de soporte pertenece. Las categorías son: ACCOUNT, CANCEL, CONTACT, DELIVERY, FEEDBACK, INVOICE, ORDER, PAYMENT, REFUND, SHIPPING y SUBSCRIPTION.

2.2 Objetivo general

Diseñar e implementar un motor de inferencia basado en el algoritmo Naïve Bayes Multinomial que clasifique automáticamente solicitudes de soporte al cliente en categorías predefinidas, integrándolo con una interfaz web funcional para su uso en tiempo real.

2.3 Objetivos específicos

- Preprocesar datos textuales aplicando limpieza, tokenización, eliminación de stopwords y stemming.
- Construir un vocabulario mediante la técnica de Bag of Words a partir del corpus de entrenamiento.
- Implementar manualmente el algoritmo Naïve Bayes Multinomial, incluyendo el cálculo de probabilidades a priori por clase, verosimilitud con suavizado de Laplace y uso de suma de logaritmos durante la inferencia.
- Evaluar el modelo utilizando K-Folds cross validation (K=5) implementado manualmente.
- Analizar y explicar las métricas de evaluación obtenidas: Precisión, Recall, F1-Score por clase, Accuracy global, Macro F1 y Matriz de Confusión.
- Guardar y cargar el modelo entrenado desde un archivo persistente en formato JSON.
- Desarrollar una interfaz web que simule un portal de soporte al cliente para realizar predicciones en tiempo real.
- Integrar correctamente el backend (motor de inferencia) con el frontend (página web).

3. Descripción del dataset utilizado

Para el entrenamiento y evaluación del modelo se utilizó el dataset Bitext Customer Support LLM Chatbot Training Dataset, disponible en la plataforma Hugging Face. Este dataset fue seleccionado por la coherencia semántica entre las solicitudes y sus etiquetas de categoría, a diferencia de otros datasets cuyas etiquetas fueron asignadas de forma aleatoria.

Característica	**Detalle**
Fuente	Hugging Face (bitext)
Total de instancias	26,872
Número de categorías	11
Campo de entrada	instruction (texto de la solicitud)
Campo de salida	category (etiqueta de clase)

3.1 Categorías del dataset

El dataset contiene once categorías de clasificación que representan los departamentos típicos de un sistema de soporte al cliente:

Categoría	**Descripción**
ACCOUNT	Solicitudes relacionadas con la gestión de cuentas de usuario
CANCEL	Solicitudes de cancelación de pedidos o servicios
CONTACT	Solicitudes de información de contacto o canales de comunicación
DELIVERY	Consultas sobre el estado o problemas con entregas
FEEDBACK	Comentarios, sugerencias o quejas generales del cliente
INVOICE	Solicitudes relacionadas con facturas y comprobantes
ORDER	Consultas sobre pedidos, estado de órdenes o modificaciones
PAYMENT	Problemas o consultas sobre métodos y estados de pago
REFUND	Solicitudes de reembolso o devolución de dinero
SHIPPING	Consultas sobre envíos, tiempos de entrega y rastreo
SUBSCRIPTION	Gestión de suscripciones, renovaciones o cambios de plan

Es importante mencionar que el dataset contiene placeholders del tipo {{Order Number}}, {{Delivery Date}}, entre otros, los cuales fueron eliminados durante el preprocesamiento mediante expresiones regulares.

4. Descripción del preprocesamiento aplicado

El preprocesamiento del texto es una etapa fundamental para transformar las solicitudes en bruto en representaciones numéricas que el clasificador pueda procesar. Se implementó un pipeline secuencial con las siguientes etapas:

4.1 Limpieza del texto

La primera etapa consiste en normalizar el texto aplicando tres transformaciones principales. Primero, se convierte todo el texto a minúsculas para eliminar diferencias por capitalización. Segundo, se eliminan los placeholders con formato {{...}} presentes en el dataset mediante una expresión regular. Tercero, se eliminan todos los caracteres que no sean letras ni espacios, removiendo números, signos de puntuación y caracteres especiales.

4.2 Tokenización

La tokenización consiste en dividir el texto limpio en unidades individuales llamadas tokens. En esta implementación se utilizó la función split() de Python, que separa el texto por espacios en blanco, generando una lista de palabras individuales.

4.3 Eliminación de stopwords

Las stopwords son palabras de alta frecuencia que no aportan información semántica relevante para la clasificación, como artículos, preposiciones y conjunciones (the, is, about, a, an, etc.). Se utilizó la lista de stopwords en inglés proporcionada por la biblioteca NLTK. Al eliminar estas palabras, se reduce el tamaño del vocabulario y se mejora la capacidad discriminativa del modelo.

4.4 Stemming

El stemming es el proceso de reducir cada palabra a su raíz morfológica. Por ejemplo, las palabras *cancelling*, *cancelled* y *cancellation* se reducen a *cancel*. Se utilizó el algoritmo PorterStemmer de NLTK. Esta técnica permite agrupar variantes morfológicas de una misma palabra bajo un único token, reduciendo el vocabulario y fortaleciendo las señales estadísticas del clasificador.

4.5 Construcción del vocabulario (Bag of Words)

Una vez procesados todos los textos, se construyó un diccionario de vocabulario que asigna un índice numérico a cada palabra única encontrada en el corpus. El vocabulario resultante contiene 2,506 términos únicos. Cada documento se representa entonces como un vector de frecuencias de longitud igual al tamaño del vocabulario, donde cada posición indica cuántas veces aparece la palabra correspondiente en el documento. Esta representación se conoce como Bag of Words (BoW).

4.6 Resumen del pipeline

Etapa	Herramienta	Resultado
Limpieza	Expresiones regulares (re)	Texto en minúsculas sin caracteres especiales
Tokenización	Python split()	Lista de tokens individuales
Stopwords	NLTK (english)	Tokens sin palabras vacías
Stemming	NLTK PorterStemmer	Tokens reducidos a raíz morfológica
Vocabulario	Implementación propia	Diccionario de 2,506 términos únicos
Vectorización	Implementación propia	Vectores BoW de 2,506 dimensiones

5. Explicación matemática del algoritmo Naïve Bayes

5.1 Fundamento del algoritmo

El clasificador Naïve Bayes se basa en el Teorema de Bayes, que establece la relación entre la probabilidad condicional de una clase dado un conjunto de características observadas. La idea central es calcular, para cada clase posible, la probabilidad de que un documento pertenezca a dicha clase dado su contenido, y seleccionar la clase con mayor probabilidad.

El Teorema de Bayes se expresa como:

$$P(\text{clase} \mid \text{documento}) = P(\text{documento} \mid \text{clase}) \times P(\text{clase}) / P(\text{documento})$$

Dado que P(documento) es constante para todas las clases al evaluar un mismo documento, la regla de decisión se simplifica a seleccionar la clase que maximice el producto P(documento | clase) × P(clase).

5.2 Modelo Multinomial

En la variante Multinomial de Naïve Bayes, el documento se representa como un vector de frecuencias de palabras (Bag of Words). Se asume que las palabras son condicionalmente independientes dada la clase (esta es la suposición "naïve" del algoritmo). Bajo esta suposición, la probabilidad del documento dado la clase se descompone como el producto de las probabilidades individuales de cada palabra:

$$P(\text{doc} \mid \text{clase}) = P(w1 \mid \text{clase}) \times P(w2 \mid \text{clase}) \times \ldots \times P(wn \mid \text{clase})$$

5.3 Probabilidad a priori

La probabilidad a priori de cada clase se estima a partir de la proporción de documentos pertenecientes a esa clase en el conjunto de entrenamiento:

$$P(\text{clase}) = \text{Número de documentos en la clase} / \text{Total de documentos}$$

En la implementación, se almacena directamente el logaritmo de esta probabilidad para facilitar los cálculos posteriores.

5.4 Suavizado de Laplace (Laplace Smoothing)

Un problema fundamental del modelo Naïve Bayes es que si una palabra del documento de prueba no apareció en ningún documento de entrenamiento para una clase determinada, su probabilidad condicional sería cero, anulando el producto completo. Para resolver este problema se aplica el suavizado de Laplace, que añade un factor de suavizado (α) al conteo de cada palabra:

$$P(\text{palabra} \mid \text{clase}) = (\text{conteo}(\text{palabra}, \text{clase}) + \alpha) / (\text{total palabras en clase} + \alpha \times |V|)$$

Donde |V| es el tamaño del vocabulario y α es el parámetro de suavizado (en esta implementación, α = 1.0). De esta forma, ninguna probabilidad es exactamente cero.

5.5 Suma de logaritmos

Multiplicar muchas probabilidades pequeñas puede causar underflow numérico, donde el resultado se redondea a cero por las limitaciones de precisión del punto flotante. Para evitar este problema, se aplica el logaritmo natural a las probabilidades, transformando el producto en una suma:

$$\log P(\text{clase} \mid \text{doc}) = \log P(\text{clase}) + \sum\left(\text{frecuencia}(w_i) \times \log P(w_i \mid \text{clase}) \right)$$

La clase predicha es aquella cuya suma de logaritmos sea la mayor. Esta técnica preserva el orden relativo de las probabilidades (la función logaritmo es monótona creciente) mientras garantiza estabilidad numérica.

6. Explicación e interpretación de las métricas de evaluación

6.1 Precisión (Precision)

La Precisión mide la proporción de predicciones positivas que fueron correctas. Para una clase determinada, se calcula como:

$$\text{Precisión} = VP / (VP + FP)$$

Donde VP son los Verdaderos Positivos y FP son los Falsos Positivos. Una precisión alta indica que el modelo rara vez confunde otras categorías con la clase en cuestión.

6.2 Recall

El Recall mide la proporción de instancias positivas reales que fueron identificadas correctamente:

$$\text{Recall} = VP / (VP + FN)$$

Donde FN son los Falsos Negativos. Un recall alto indica que el modelo identifica la mayoría de las instancias pertenecientes a esa clase.

6.3 F1-Score

El F1-Score es la media armónica entre Precisión y Recall:

$$F1 = 2 \times (\text{Precisión} \times \text{Recall}) / (\text{Precisión} + \text{Recall})$$

6.4 Accuracy (Exactitud global)

La Accuracy mide la proporción total de predicciones correctas sobre el total de instancias:

$$\text{Accuracy} = \text{Predicciones correctas} / \text{Total de predicciones}$$

6.5 Macro F1

El Macro F1 es el promedio aritmético de los F1-Score de todas las clases, tratando cada clase con igual importancia:

$$\text{Macro F1} = (1/N) \times \sum(\text{F1 de cada clase})$$

6.6 Matriz de Confusión

La Matriz de Confusión es una tabla que visualiza el desempeño del clasificador mostrando, para cada clase real (fila), cuántas instancias fueron asignadas a cada clase predicha (columna). Los valores en la diagonal principal representan las clasificaciones correctas.

6.7 Resultados por clase

Categoría	Precisión	Recall	F1-Score
ACCOUNT	0.9978	0.9982	0.9980
CANCEL	1.0000	0.9958	0.9979
CONTACT	0.9995	0.9980	0.9987
DELIVERY	0.9799	0.9283	0.9534
FEEDBACK	0.9960	0.9990	0.9975
INVOICE	0.9990	0.9910	0.9950
ORDER	0.9766	0.9925	0.9845
PAYMENT	0.9955	0.9975	0.9965
REFUND	0.9987	0.9957	0.9972
SHIPPING	0.9680	0.9970	0.9822
SUBSCRIPTION	0.9980	0.9990	0.9985

Métrica global	Valor
Accuracy Global	0.9912
Macro F1-Score	0.9909

6.8 Matriz de Confusión

Real/Pred	ACCOUNT	CANCEL	CONTACT	DELIVERY	FEEDBACK	INVOICE	ORDER	PAYMENT	REFUND	SHIPPING	SUBSCR.
ACCOUNT	5975	0	1	0	0	0	1	8	0	0	1
CANCEL	0	946	0	1	0	0	2	0	1	0	0
CONTACT	0	0	1995	0	2	0	1	1	0	0	0
DELIVERY	0	0	0	1851	0	0	84	0	0	59	0
FEEDBACK	0	0	0	0	1995	0	1	0	0	1	0
INVOICE	0	0	0	10	5	1981	2	0	0	1	0
ORDER	5	0	0	15	1	2	3958	0	3	4	0
PAYMENT	3	0	0	1	0	0	1	1993	0	0	0
REFUND	0	0	0	11	0	0	1	0	2979	0	1
SHIPPING	5	0	0	0	0	0	1	0	0	1964	0
SUBSCR.	0	0	0	0	0	0	1	0	0	0	998

6.9 Análisis crítico

Clases con mejor desempeño: Las categorías CONTACT (F1 = 0.9987), SUBSCRIPTION (F1 = 0.9985), ACCOUNT (F1 = 0.9980) y CANCEL (F1 = 0.9979)

presentan métricas casi perfectas. Esto se debe a que utilizan vocabulario altamente específico y diferenciado de las demás categorías.

Clases con menor desempeño: Las categorías DELIVERY (F1 = 0.9534), SHIPPING (F1 = 0.9822) y ORDER (F1 = 0.9845) presentan las métricas más bajas. La Matriz de Confusión revela que DELIVERY se confunde principalmente con ORDER (84 instancias) y SHIPPING (59 instancias). Esto es esperable, ya que las solicitudes sobre entregas comparten vocabulario con solicitudes sobre pedidos y envíos. Las confusiones se concentran en el triángulo DELIVERY-SHIPPING-ORDER, lo cual es coherente con la superposición semántica natural de estos dominios.

7. Resultados obtenidos con K-Folds y análisis de varianza

7.1 Validación cruzada con K=5

La validación cruzada K-Folds es una técnica de evaluación que permite estimar el desempeño del modelo de forma robusta, utilizando todo el dataset tanto para entrenamiento como para validación. El dataset de 26,872 instancias se dividió en 5 particiones (folds) de forma aleatoria. En cada iteración, se utilizaron 4 particiones para entrenamiento y 1 para validación, rotando el fold de validación en cada iteración. Este proceso se implementó completamente de forma manual, sin utilizar bibliotecas externas.

Fold	**Accuracy**
Fold 1	0.9913
Fold 2	0.9922
Fold 3	0.9920
Fold 4	0.9911
Fold 5	0.9894

7.2 Análisis de varianza entre folds

Estadístico	**Valor**
Accuracy promedio	0.9912
Varianza entre folds	0.000001
Diferencia (máximo - mínimo)	0.0028

La varianza obtenida entre los cinco folds es extremadamente baja (0.000001), lo que indica que el modelo presenta un comportamiento muy estable y consistente independientemente de la partición utilizada para validación. La diferencia entre el fold con mejor rendimiento (Fold 2: 0.9922) y el de menor rendimiento (Fold 5: 0.9894) es de apenas 0.0028, lo cual confirma que no existe sobreajuste significativo.

Estos resultados permiten concluir que el modelo generaliza de forma robusta y que el rendimiento observado no depende de una partición particular de los datos.

8. Diagramas

8.1 Diagrama de Arquitectura de la Solución

El sistema se compone de tres capas principales:

Capa	Componente	Tecnología
Presentación	Interfaz web (formulario de tickets)	HTML, CSS, JavaScript
Lógica	Servidor web y API REST	FastAPI (Python)
Lógica	Pipeline de preprocesamiento	Python, NLTK, re
Lógica	Motor de inferencia Naïve Bayes	Python (impl. propia)
Datos	Modelo entrenado	JSON (naive_bayes_model.json)
Datos	Vocabulario	JSON (vocab.json)
Datos	Dataset original	Hugging Face (Bitext)

El flujo de una predicción inicia cuando el usuario ingresa una solicitud en el formulario web. El frontend envía una petición POST al endpoint /predict del servidor FastAPI. El backend procesa el texto a través del pipeline de preprocesamiento, lo convierte en un vector Bag of Words y lo pasa al motor de inferencia Naïve Bayes. El modelo retorna la categoría predicha en formato JSON.

8.2 Diagrama de Casos de Uso

Caso de uso	Descripción
Ingresar solicitud	El usuario escribe el asunto y la descripción de su problema
Clasificar ticket	El sistema clasifica automáticamente la solicitud en una categoría
Visualizar resultado	El usuario observa la categoría predicha por el modelo
Crear nuevo ticket	El usuario limpia el formulario y genera un nuevo ID de ticket

8.3 Diagrama de Flujo General

Paso	Acción	Detalle
1	Ingreso de texto	El usuario escribe asunto y descripción en el formulario

2	Envío al servidor	El frontend envía POST /predict con el texto concatenado
3	Limpieza	Se eliminan placeholders, caracteres especiales y se pasa a minúsculas
4	Tokenización	Se divide el texto en tokens individuales
5	Stopwords	Se remueven palabras sin carga semántica
6	Stemming	Se reducen los tokens a su raíz morfológica
7	Vectorización BoW	Se convierte la lista de tokens en un vector de frecuencias
8	Inferencia	El modelo calcula log-probabilidades y selecciona la clase máxima
9	Respuesta JSON	El servidor retorna la categoría predicha
10	Visualización	El frontend muestra la categoría al usuario

8.4 Diagrama de Componentes

Componente	Archivo(s)	Responsabilidad
Carga de datos	src/utils/load_data.py	Descarga del dataset desde Hugging Face
Limpieza	src/preprocessing/clean_text.py	Normalización y limpieza del texto
Tokenización	src/preprocessing/tokenizer.py	Tokenización, stopwords y stemming
Vectorización	src/model/vectorizer.py	Vocabulario y vectores BoW
Pipeline	src/pipeline.py	Orquestación del preprocesamiento
Clasificador NB	src/model/naive_bayes.py	Entrenamiento, predicción, guardado y carga
Métricas	src/evaluation/metrics.py	Accuracy, Precision, Recall, F1, MC
K-Folds	src/evaluation/k_folds.py	Validación cruzada manual
Entrenamiento	src/train.py	Ejecución de K-Folds y entrenamiento final
Servidor web	web/app.py	API REST con FastAPI
Frontend	web/templates/index.html	Interfaz de usuario HTML
Estilos	web/static/css/styles.css	Diseño visual de la interfaz
Lógica cliente	web/static/js/app.js	Interacción y llamadas al API

8.5 Diagrama de Secuencias

El diagrama de secuencias describe la interacción entre los componentes del sistema durante una clasificación de ticket:

Paso	Origen	Destino	Acción
1	Usuario	Frontend (JS)	Completa formulario y presiona Clasificar
2	Frontend (JS)	FastAPI (/predict)	POST con JSON {texto: ...}
3	FastAPI	pipeline.py	Llama preprocess_text(texto)
4	pipeline.py	clean_text.py	Limpieza del texto

5	pipeline.py	tokenizer.py	Tokenización, stopwords y stemming
6	FastAPI	vectorizer.py	Convierte tokens a vector BoW
7	FastAPI	naive_bayes.py	Llama predict(vector)
8	naive_bayes.py	FastAPI	Retorna categoría predicha
9	FastAPI	Frontend (JS)	Responde JSON {categoria: ...}
10	Frontend (JS)	Usuario	Muestra la categoría en pantalla

9. Evidencias de funcionamiento

9.1 Salida del entrenamiento y evaluación

A continuación se presenta la salida real obtenida al ejecutar el script de entrenamiento (train.py):

```
[INFO] Ejecutando 5-Folds Cross Validation...
 -> Evaluando Fold 1/5...
 -> Evaluando Fold 2/5...
 -> Evaluando Fold 3/5...
 -> Evaluando Fold 4/5...
 -> Evaluando Fold 5/5...

--- RESULTADOS K-FOLDS ---
Accuracies obtenidas : [0.9913, 0.9922, 0.992, 0.9911, 0.9894]
Promedio general     : 0.9912
Varianza de Folds    : 0.000001
Diferencia (Max-Min) : 0.0028

--- REPORTE DE MÉTRICAS ---
Accuracy Global : 0.9912
Macro F1-Score  : 0.9909

Clase               | Precision  | Recall     | F1-Score
-------------------------------------------------------
ACCOUNT             | 0.9978     | 0.9982     | 0.9980
CANCEL              | 1.0000     | 0.9958     | 0.9979
CONTACT             | 0.9995     | 0.9980     | 0.9987
DELIVERY            | 0.9799     | 0.9283     | 0.9534
FEEDBACK            | 0.9960     | 0.9990     | 0.9975
INVOICE             | 0.9990     | 0.9910     | 0.9950
ORDER               | 0.9766     | 0.9925     | 0.9845
PAYMENT             | 0.9955     | 0.9975     | 0.9965
REFUND              | 0.9987     | 0.9957     | 0.9972
SHIPPING            | 0.9680     | 0.9970     | 0.9822
SUBSCRIPTION        | 0.9980     | 0.9990     | 0.9985
-------------------------------------------------------
```

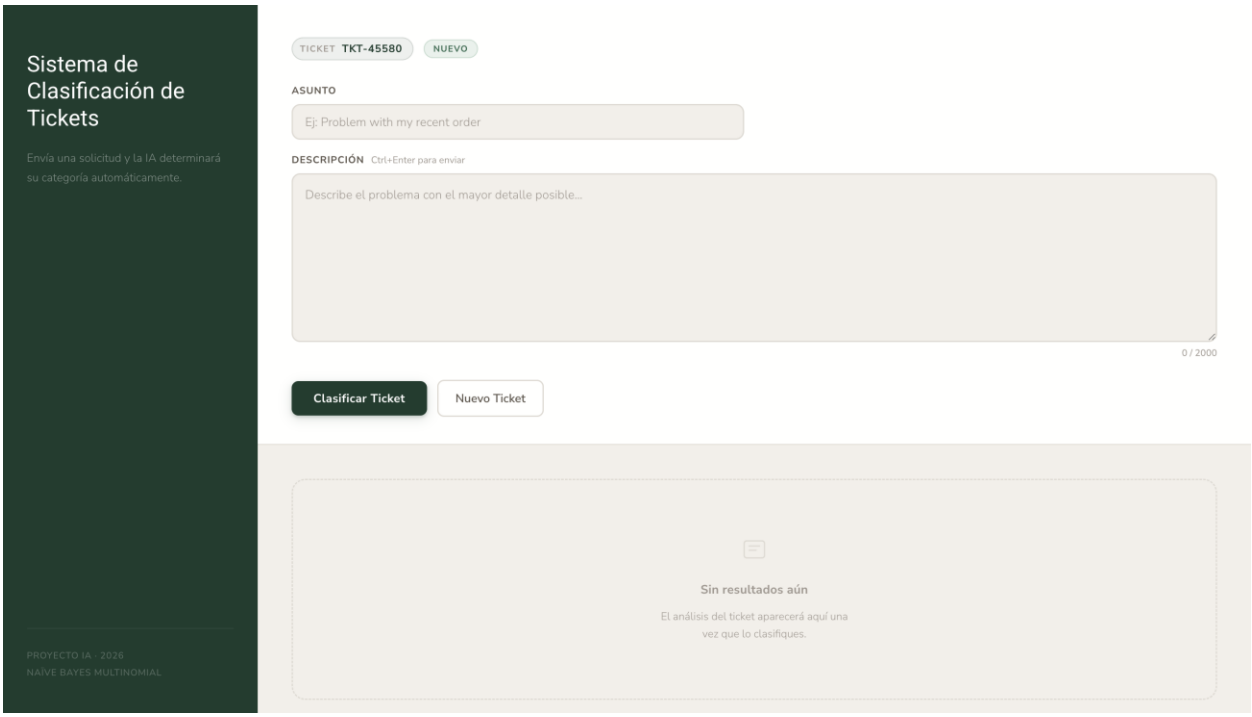
```
--- MATRIZ DE CONFUSIÓN ---
Filas = Real, Columnas = Predicción

Real/Pred      ACCOUNT   CANCEL  CONTACT DELIVERY FEEDBACK  INVOICE    ORDER  PAYMENT   REFUND SHIPPING SUBSCRIP
-----------------------------------------------------------------------------------------------------------------
ACCOUNT      |    5975        0        1        0        0        0        1        8        0        0        1
CANCEL       |       0      946        0        1        0        0        2        0        1        0        0
CONTACT      |       0        0     1995        0        2        0        1        1        0        0        0
DELIVERY     |       0        0        0     1851        0        0       84        0        0       59        0
FEEDBACK     |       0        0        0        0     1995        0        1        0        0        1        0
INVOICE      |       0        0        0       10        5     1981        2        0        0        1        0
ORDER        |       5        0        0       15        1        2     3958        0        3        4        0
PAYMENT      |       3        0        0        1        0        0        1     1993        0        0        0
REFUND       |       0        0        0       11        0        0        1        0     2979        0        1
SHIPPING     |       5        0        0        0        0        0        1        0        0     1964        0
SUBSCRIPTI   |       0        0        0        0        0        0        1        0        0        0      998

[INFO] Entrenando modelo final definitivo con el 100% de las instancias...

[ÉXITO] Modelo de IA exportado satisfactoriamente hacia: models/naive_bayes_model.json
(.venv) erickrivas@MacBook-Air-de-Erick-2 proyecto IA %
```

9.2 Interfaz web

La aplicación web fue desarrollada con FastAPI en el backend y HTML/CSS/JavaScript en el frontend. La interfaz simula un sistema de tickets de soporte donde el usuario puede:

- Visualizar un ID de ticket autogenerado (formato TKT-XXXXX).
- Ingresar un asunto (subject) con un máximo de 150 caracteres.
- Escribir una descripción detallada con un máximo de 2,000 caracteres.
- Presionar el botón "Clasificar Ticket" para obtener la predicción.
- Observar la categoría predicha mostrada en tiempo real.

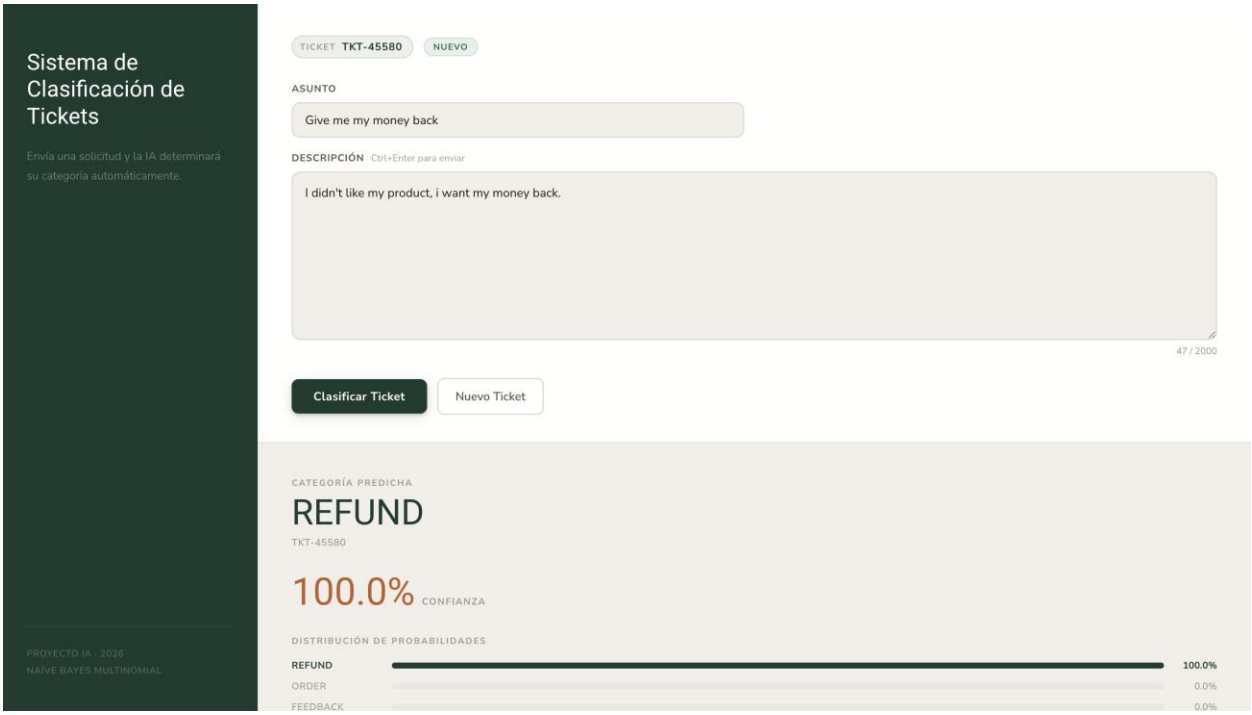
- Crear un nuevo ticket limpiando el formulario con el botón "Nuevo Ticket".



9.3 Ejemplo de clasificación

Campo	Contenido
Ticket ID	TKT-45580(autogenerado)
Asunto	Give me my money back
Descripción	I didn't like my product, i want my money back.
Categoría predicha	REFUND

El modelo clasificó correctamente esta solicitud como REFUND, dado que el contenido hace referencia a que el cliente quiere su dinero de regreso.



10. Conclusiones y aprendizajes

10.1 Conclusiones

El proyecto demostró que el algoritmo Naïve Bayes Multinomial, implementado desde cero, es capaz de alcanzar un rendimiento sobresaliente en la tarea de clasificación de solicitudes de soporte al cliente, con un Accuracy global de 99.12% y un Macro F1-Score de 99.09%. Estos resultados validan la efectividad del enfoque probabilístico combinado con un preprocesamiento adecuado del texto.

La integración exitosa del motor de inferencia con la aplicación web demuestra que el sistema es viable como solución práctica para el enrutamiento automático de tickets en un entorno de soporte real. El modelo serializado en formato JSON permite su carga rápida sin necesidad de reentrenamiento.

Las confusiones observadas entre las categorías DELIVERY, SHIPPING y ORDER son coherentes con la superposición semántica natural de estos dominios y representan una limitación inherente del modelo basado en frecuencia de términos, no un defecto de la implementación.

10.2 Aprendizajes

- La implementación manual del algoritmo obligó a comprender en profundidad cada componente matemático: el Teorema de Bayes, el suavizado de Laplace y la aritmética en espacio logarítmico.
- El preprocesamiento del texto tiene un impacto directo y significativo en la calidad del modelo. La limpieza de placeholders, la eliminación de stopwords y el stemming contribuyeron a reducir el vocabulario de forma efectiva.
- La validación cruzada K-Folds proporcionó una estimación confiable del rendimiento del modelo y permitió verificar la ausencia de sobreajuste.
- La Matriz de Confusión resultó ser la herramienta más valiosa para el análisis cualitativo de los errores del clasificador, permitiendo identificar patrones de confusión específicos.
- La integración backend-frontend con FastAPI demostró ser una solución limpia y eficiente para exponer el modelo como servicio web, manteniendo una separación clara de responsabilidades.
- La serialización del modelo en JSON plano, aunque menos eficiente que formatos binarios como pickle, ofrece la ventaja de ser completamente inspeccionable y portátil.